

Modern Operating Systems — Chapter 6 Deadlocks Exercises and Answers

Andrew S. Tanenbaum and Herbert Bos

Contents

Chapter 6: Deadlocks	2
Exercise 1	2
Exercise 2	2
Exercise 3	3
Exercise 4	3
Exercise 5	3
Exercise 6	4
Exercise 7	5
Exercise 8	5
Exercise 9	6
Exercise 10	6
Exercise 11	7
Exercise 12	7
Exercise 13	8
Exercise 14	8
Exercise 15	9

Exercise 16	9
Exercise 17	10
Exercise 18	10
Exercise 19	10
Exercise 20	11
Exercise 21	11
Exercise 22	12
Exercise 23	12
Exercise 24	13
Exercise 25	13
Exercise 26	14
Exercise 27	14
Exercise 28	15
Exercise 29	15
Exercise 30	16
Exercise 31	16
Exercise 32	17
Exercise 33	17
Exercise 34	18
Exercise 35	18
Exercise 36	19

Exercise 37	19
Exercise 38	20
Exercise 39	21
Exercise 40	22
Exercise 41	23
Exercise 42	25
Exercise 43	26
Exercise 44	27

Chapter 6: Deadlocks

Source note. Questions were extracted from the Chapter 6 problem section of *Modern Operating Systems*, 4th edition. Answers 1–38 were matched to the Chapter 6 answer section of the provided `MOS_4th_answer.pdf`. Exercises 39–44 had no matching answer in that answer section, so their answers are clearly labeled as prepared technical answers rather than answer-manual text. Additional math explanations are separately labeled where added.

Exercise 1

Question

Give an example of a deadlock taken from politics.

Answer

In the U.S., consider a presidential election in which three or more candidates are trying for the nomination of some party. After all the primary elections are finished, when the delegates arrive at the party convention, it could happen that no candidate has a majority and that no delegate is willing to change his or her vote. This is a deadlock. Each candidate has some resources (votes) but needs more to get the job done. In countries with multiple political parties in the parliament, it could happen that each party supports a different version of the annual budget and that it is impossible to assemble a majority to pass the budget. This is also a deadlock.

Source: Answer manual.

Exercise 2

Question

Students working at individual PCs in a computer laboratory send their files to be printed by a server that spools the files on its hard disk. Under what conditions may a deadlock occur if the disk space for the print spool is limited? How may the deadlock be avoided?

Answer

Disk space on the spooling partition is a finite resource. Every block that comes in de facto claims a resource and every new one arriving wants more resources. If the spooling space is, say, 10 MB and the first half of ten 2-MB jobs arrive, the disk will be full and no more blocks can be stored so we have a deadlock. The deadlock can be avoided by allowing a job to start printing before it is fully spooled and reserving the space thus released for the rest of that job. In this way, one job will actually print to completion, then the next one can do the same thing. If jobs cannot start printing until they are fully spooled, deadlock is

possible.

Source: Answer manual.

Exercise 3

Question

In the preceding question, which resources are preemptable and which are nonpreemptable?

Answer

The printer is nonpreemptable; the system cannot start printing another job until the previous one is complete. The spool disk is preemptable; you can delete an incomplete file that is growing too large and have the user send it later, assuming the protocol allows that.

Source: Answer manual.

Exercise 4

Question

In Fig. 6-1 the resources are returned in the reverse order of their acquisition. Would giving them back in the other order be just as good?

Answer

Yes. It does not make any difference whatsoever.

Source: Answer manual.

Exercise 5

Question

The four conditions (mutual exclusion, hold and wait, no preemption and circular wait) are necessary for a resource deadlock to occur. Give an example to show that these conditions are not sufficient for a resource deadlock to occur. When are these conditions sufficient for a resource deadlock to occur?

Answer

Suppose that there are three processes, A, B and C, and two resource types, R and S. Further assume that there are one instance of R and two instance of S. Consider the following execution scenario: A requests R and gets it; B requests S and gets; C requests S and gets it (there are two instances of S); B requests R and is blocked; A requests S and is blocked. At this stage all four conditions hold. However, there is no deadlock. When C finishes, one instance of S is released that is allocated to A. Now A can complete its execution and release R that can be allocated to B, which can then complete its execution. These four conditions are enough if there is one resource of each type.

Source: Answer manual.

Exercise 6

Question

City streets are vulnerable to a circular blocking condition called gridlock, in which intersections are blocked by cars that then block cars behind them that then block the cars that are trying to enter the previous intersection, etc. All intersections around a city block are filled with vehicles that block the oncoming traffic in a circular manner. Gridlock is a resource deadlock and a problem in competition synchronization. New York City's prevention algorithm, called "don't block the box," prohibits cars from entering an intersection unless the space following the intersection is also available. Which prevention algorithm is this? Can you provide any other prevention algorithms for gridlock?

Answer

"Don't block the box" is a pre-allocation strategy, negating the hold and wait deadlock precondition, since we assume that cars can enter the street space following the intersection, thus freeing the intersection. Another strategy might allow cars to temporarily pull into garages and release enough space to clear the gridlock. Some cities have a traffic control policy to shape traffic; as city streets become more congested, traffic supervisors adjust the settings for red lights in order to throttle traffic entering heavily congested areas. Lighter traffic ensures less competition over resources and thus lowers the probability of gridlock occurring.

Source: Answer manual.

Exercise 7

Question

Suppose four cars each approach an intersection from four different directions simultaneously. Each corner of the intersection has a stop sign. Assume that traffic regulations require that when two cars approach adjacent stop signs at the same time, the car on the left must yield to the car on the right. Thus, as four cars each drive up to their individual stop signs, each waits (indefinitely) for the car on the left to proceed. Is this anomaly a communication deadlock? Is it a resource deadlock?

Answer

The above anomaly is not a communication deadlock since these cars are independent of each other and would drive through the intersection with a minimal delay if no competition occurred. It is not a resource deadlock, since no car is holding a resource that is requested by another car. Nor would the mechanisms of resource pre-allocation or of resource preemption assist in controlling this anomaly. This anomaly is one of competition synchronization, however, in which cars are waiting for resources in a circular chain and traffic throttling may be an effective strategy for control. To distinguish from resource deadlock, this anomaly might be termed a “scheduling deadlock.” A similar deadlock could occur following a law that required two trains merging onto a shared railroad track to wait for the other to proceed. Note that a policeman signaling one of the competing cars or trains to proceed (and not the others) can break this dead state without rollback or any other overhead.

Source: Answer manual.

Exercise 8

Question

Is it possible that a resource deadlock involves multiple units of one type and a single unit of another? If so, give an example.

Answer

It is possible that one process holds some or all of the units of one resource type and requests another resource type, while another process holds the second resource while requesting the available units of the first resource type. If no other process can release units of the first resource type and the resource cannot be preempted or used concurrently, the system is deadlocked. For example, two processes are both allocated memory cells in a real memory system. (We assume that swapping of pages or processes is not supported, while dynamic requests for memory are supported.) The first process locks another resource - perhaps a data cell. The second process requests the locked data and is blocked. The first

process needs more memory in order to execute the code to release the data. Assuming that no other processes in the system can complete and release memory cells, a deadlock exists in the system.

Source: Answer manual.

Exercise 9

Question

Fig. 6-3 shows the concept of a resource graph. Do illegal graphs exist, that is, graphs that structurally violate the model we have used of resource usage? If so, give an example of one.

Answer

Yes, illegal graphs exist. We stated that a resource may only be held by a single process. An arc from a resource square to a process circle indicates that the process owns the resource. Thus, a square with arcs going from it to two or more processes means that all those processes hold the resource, which violates the rules. Consequently, any graph in which multiple arcs leave a square and end in different circles violates the rules unless there are multiple copies of the resources. Arcs from squares to squares or from circles to circles also violate the rules.

Source: Answer manual.

Exercise 10

Question

Consider Fig. 6-4. Suppose that in step (o) C requested S instead of requesting R. Would this lead to deadlock? Suppose that it requested both S and R.

Answer

Neither change leads to deadlock. There is no circular wait in either case.

Source: Answer manual.

Exercise 11

Question

Suppose that there is a resource deadlock in a system. Give an example to show that the set of processes deadlocked can include processes that are not in the circular chain in the corresponding resource allocation graph.

Answer

Consider three processes, A, B and C and two resources R and S. Suppose A is waiting for I that is held by B, B is waiting for S held by A, and C is waiting for R held by A. All three processes, A, B and C are deadlocked. However, only A and B belong to the circular chain.

Source: Answer manual.

Exercise 12

Question

In order to control traffic, a network router, A periodically sends a message to its neighbor, B, telling it to increase or decrease the number of packets that it can handle. At some point in time, Router A is flooded with traffic and sends B a message telling it to cease sending traffic. It does this by specifying that the number of bytes B may send (A's window size) is 0. As traffic surges decrease, A sends a new message, telling B to restart transmission. It does this by increasing the window size from 0 to a positive number. That message is lost. As described, neither side will ever transmit. What type of deadlock is this?

Answer

This is clearly a communication deadlock, and can be controlled by having A time out and retransmit its enabling message (the one that increases the window size) after some period of time (a heuristic). It is possible, however, that B has received both the original and the duplicate message. No harm will occur if the update on the window size is given as an absolute value and not as a differential. Sequence numbers on such messages are also effective to detect duplicates.

Source: Answer manual.

Exercise 13

Question

The discussion of the ostrich algorithm mentions the possibility of process-table slots or other system tables filling up. Can you suggest a way to enable a system administrator to recover from such a situation?

Answer

A portion of all such resources could be reserved for use only by processes owned by the administrator, so he or she could always run a shell and programs needed to evaluate a deadlock and make decisions about which processes to kill to make the system usable again.

Source: Answer manual.

Exercise 14

Question

Consider the following state of a system with four processes, P1, P2, P3, and P4, and five types of resources, RS1, RS2, RS3, RS4, and RS5: $0\ 0\ 0\ 2\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 0$ $E = (24144)$ $A = (01021)$ $C = R = 1\ 2\ 1\ 0\ 0\ 1\ 1\ 0\ 1\ 1\ 0\ 2\ 1\ 0\ 1\ 0\ 2\ 1\ 0\ 2\ 0\ 3\ 1\ 0\ 2\ 1\ 1\ 0$ Using the deadlock detection algorithm described in Section 6.4.2, show that there is a deadlock in the system. Identify the processes that are deadlocked.

Answer

First, the set of unmarked processes, $P = (P1\ P2\ P3\ P4)$ $R1$ is not less than or equal to A $R2$ is less than A ; Mark $P2$; $A = (0\ 2\ 0\ 3\ 1)$; $P = (P1\ P3\ P4)$ $R1$ is not less than or equal to A $R3$ is equal to A ; Mark $P3$; $A = (0\ 2\ 0\ 3\ 2)$; $P = (P1\ P4)$ $R1$ is not less than or equal to A $R4$ is not less than or equal to A So, processes $P1$ and $P4$ remain unmarked. They are deadlocked.

Source: Answer manual.

Math explanation. In the deadlock-detection algorithm, a process can be marked if its request vector is componentwise no larger than the current available vector. Starting with the available vector $A = (0, 1, 0, 2, 1)$, compare each unmarked process request R_i with A . Process P_2 can finish first, so add its current allocation back to A , giving $A = (0, 2, 0, 3, 1)$. Then P_3 can finish, giving $A = (0, 2, 0, 3, 2)$. At that point neither P_1 nor P_4 has $R_i \leq A$, so no further progress is possible. The unmarked processes, P_1 and P_4 , are the deadlocked ones.

Source: Prepared from textbook concepts.

Exercise 15

Question

Explain how the system can recover from the deadlock in previous problem using

- (a) recovery through preemption.
- (b) recovery through rollback.
- (c) recovery through killing processes.

Answer

Recovery through preemption: After processes P2 and P3 complete, process P1 can be forced to preempt 1 unit of RS3. This will make $A = (0 \ 2 \ 1 \ 3 \ 2)$, and allow process P4 to complete. Once P4 completes and release its resources P1 may complete. Recovery through rollback: Rollback P1 to the state checkpointed before it acquired RS3. Recovery through killing processes: Kill P1.

Source: Answer manual.

Exercise 16

Question

Suppose that in Fig. 6-6 $C_{ij} + R_{ij} > E_j$ for some i . What implications does this have for the system?

Answer

The process is asking for more resources than the system has. There is no conceivable way it can get these resources, so it can never finish, even if no other processes want any resources at all.

Source: Answer manual.

Math explanation. The inequality $C_{ij} + R_{ij} > E_j$ says that process i already holds C_{ij} units of resource class j , but is requesting enough additional units that its total need would exceed the total number E_j that exists in the whole system. Even if every other process released all units of that resource, the request could not be satisfied.

Source: Prepared from textbook concepts.

Exercise 17

Question

All the trajectories in Fig. 6-8 are horizontal or vertical. Can you envision any circumstances in which diagonal trajectories are also possible?

Answer

If the system had two or more CPUs, two or more processes could run in parallel, leading to diagonal trajectories.

Source: Answer manual.

Exercise 18

Question

Can the resource trajectory scheme of Fig. 6-8 also be used to illustrate the problem of deadlocks with three processes and three resources? If so, how can this be done? If not, why not?

Answer

Yes. Do the whole thing in three dimensions. The z-axis measures the number of instructions executed by the third process.

Source: Answer manual.

Exercise 19

Question

In theory, resource trajectory graphs could be used to avoid deadlocks. By clever scheduling, the operating system could avoid unsafe regions. Is there a practical way of actually doing this?

Answer

The method can only be used to guide the scheduling if the exact instant at which a resource is going to be claimed is known in advance. In practice, this is rarely the case.

Source: Answer manual.

Exercise 20

Question

Can a system be in a state that is neither deadlocked nor safe? If so, give an example. If not, prove that all states are either deadlocked or safe.

Answer

There are states that are neither safe nor deadlocked, but which lead to deadlocked states. As an example, suppose we have four resources: tapes, plotters, scanners, and CD-ROMs, as in the text, and three processes competing for them. We could have the following situation: Has Needs Available A: 2 0 0 0 1 0 2 0 0 1 2 1 B: 1 0 0 0 0 1 3 1 C: 0 1 2 1 1 0 1 0 This state is not deadlocked because many actions can still occur, for example, A can still get two printers. However, if each process asks for its remaining requirements, we have a deadlock.

Source: Answer manual.

Math explanation. A safe state means there exists at least one ordering of processes in which each can complete and release resources for the next. A deadlocked state means no process in the relevant set can proceed. An unsafe state lies between these: the system is not stuck yet, but no completion sequence can be guaranteed if future requests follow the declared maximum needs.

Source: Prepared from textbook concepts.

Exercise 21

Question

Take a careful look at Fig. 6-11(b). If D asks for one more unit, does this lead to a safe state or an unsafe one? What if the request came from C instead of D ?

Answer

A request from D is unsafe, but one from C is safe.

Source: Answer manual.

Math explanation. The safety test is not just "is the request currently available?" The test pretends to grant the request and then searches for a completion order. Granting D's extra unit leaves no safe completion sequence, so it is unsafe. Granting C's request still leaves a sequence in which at least one process can finish, return resources, and let the others finish.

Source: Prepared from textbook concepts.

Exercise 22

Question

A system has two processes and three identical resources. Each process needs a maximum of two resources. Is deadlock possible? Explain your answer.

Answer

The system is deadlock free. Suppose that each process has one resource. There is one resource free. Either process can ask for it and get it, in which case it can finish and release both resources. Consequently, deadlock is impossible.

Source: Answer manual.

Math explanation. With two processes and three identical resources, the worst partial-allocation case is that each process holds one resource and still needs one more. That accounts for two resources, leaving one free. Give the free resource to either process; it reaches its maximum of two, finishes, and returns both resources. Then the other process can finish. Therefore a circular wait cannot be permanent.

Source: Prepared from textbook concepts.

Exercise 23

Question

Consider the previous problem again, but now with p processes each needing a maximum of m resources and a total of r resources available. What condition must hold to make the system deadlock free?

Answer

If a process has m resources it can finish and cannot be involved in a deadlock. Therefore, the worst case is where every process has $m - 1$ resources and needs another one. If there is one resource left over, one process can finish and release all its resources, letting the rest finish too. Therefore the condition for avoiding deadlock is $r \geq p(m - 1) + 1$.

Source: Answer manual.

Math explanation. The dangerous worst case is that every one of the p processes holds $m - 1$ resources and needs one more. That consumes $p(m - 1)$ resources. If the system has at least one additional resource, one process can receive its last unit, finish, and release all m units. This starts a chain of completions. Therefore the deadlock-free condition is $r \geq p(m - 1) + 1$.

Source: Prepared from textbook concepts.

Exercise 24

Question

Suppose that process A in Fig. 6-12 requests the last tape drive. Does this action lead to a deadlock?

Answer

No. D can still finish. When it finishes, it returns enough resources to allow E (or A) to finish, and so on.

Source: Answer manual.

Exercise 25

Question

The banker's algorithm is being run in a system with m resource classes and n processes. In the limit of large m and n , the number of operations that must be performed to check a state for safety is proportional to $m^a n^b$. What are the values of a and b ?

Answer

Comparing a row in the matrix to the vector of available resources takes m operations. This step must be repeated on the order of n times to find a process that can finish and be marked as done. Thus, marking a process as done takes on the order of mn steps. Repeating the algorithm for all n processes means that the number of steps is then mn^2 . Thus, $a = 1$ and $b = 2$.

Source: Answer manual.

Math explanation. Checking whether one process's request/need vector can be satisfied requires comparing m resource classes. In the worst case, finding one process that can finish may require scanning n processes, so one successful mark costs on the order of mn . The algorithm may need to mark up to n processes, giving mn^2 . Thus the exponents in $m^a n^b$ are $a = 1$ and $b = 2$.

Source: Prepared from textbook concepts.

Exercise 26

Question

A system has four processes and five allocatable resources. The current allocation and maximum needs are as follows: Allocated Maximum Available Process A 1 0 2 1 1 1 1 2 1 3 0 0 x 1 1 Process B 2 0 1 1 0 2 2 2 1 0 Process C 1 1 0 1 0 2 1 3 1 0 Process D 1 1 1 1 0 1 1 2 2 1 What is the smallest value of x for which this is a safe state?

Answer

The needs matrix is as follows: 0 1 0 0 2 0 2 1 0 0 1 0 3 0 0 0 0 1 1 1 If x is 0, we have a deadlock immediately. If x is 1, process D can run to completion. When it is finished, the available vector is 1 1 2 2 1. Unfortunately we are now deadlocked. If x is 2, after D runs, the available vector is 1 1 3 2 1 and C can run. After it finishes and returns its resources the available vector is 2 2 3 3 1, which will allow B to run and complete, and then A to run and complete. Therefore, the smallest value of x that avoids a deadlock is 2.

Source: Answer manual.

Math explanation. The need matrix is computed as $\text{Need} = \text{Maximum} - \text{Allocated}$, component by component. The smallest value of x is the one that makes the initial available vector large enough for at least one process to complete, after which its allocation is added back to Available. The answer manual's threshold $x = 1$ means that with zero units of that resource no safe completion sequence begins, while with one unit there is a valid sequence.

Source: Prepared from textbook concepts.

Exercise 27

Question

One way to eliminate circular wait is to have rule saying that a process is entitled only to a single resource at any moment. Give an example to show that this restriction is unacceptable in many cases.

Answer

Consider a process that needs to copy a huge file from a tape to a printer. Because the amount of memory is limited and the entire file cannot fit in this memory, the process will have to loop through the following statements until the entire file has been printed: Acquire tape drive Copy the next portion of the file in memory (limited memory size) Release tape drive Acquire printer Print file from memory Release printer This will lengthen the execution time of the process. Furthermore, since the printer is released after every print step, there is no guarantee that all portions of the file will get printed on continuous pages.

Source: Answer manual.

Exercise 28

Question

Two processes, A and B, each need three records, 1, 2, and 3, in a database. If A asks for them in the order 1, 2, 3, and B asks for them in the same order, deadlock is not possible. However, if B asks for them in the order 3, 2, 1, then deadlock is possible. With three resources, there are $3!$, or six possible combinations in which each process can request them. What fraction of all the combinations is guaranteed to be deadlock free?

Answer

Suppose that process A requests the records in the order a, b, c. If process B also asks for a first, one of them will get it and the other will block. This situation is always deadlock free since the winner can now run to completion without interference. Of the four other combinations, some may lead to deadlock and some are deadlock free. The six cases are as follows: a b c deadlock free a c b deadlock free b a c possible deadlock b c a possible deadlock c a b possible deadlock c b a possible deadlock Since four of the six may lead to deadlock, there is a $1/3$ chance of avoiding a deadlock and a $2/3$ chance of getting one.

Source: Answer manual.

Math explanation. Once A's order is fixed as (1, 2, 3), B has $3! = 6$ possible orderings. If both processes acquire records in the same increasing order, one process waits at the first conflict while the other can continue to completion. Deadlock requires a cycle: for example, one process holds a low-numbered record and waits upward while the other holds a high-numbered record and waits downward.

Source: Prepared from textbook concepts.

Exercise 29

Question

A distributed system using mailboxes has two IPC primitives, send and receive. The latter primitive specifies a process to receive from and blocks if no message from that process is available, even though messages may be waiting from other processes. There are no shared resources, but processes need to communicate frequently about other matters. Is deadlock possible? Discuss.

Answer

Yes. Suppose that all the mailboxes are empty. Now A sends to B and waits for a reply, B sends to C and waits for a reply, and C sends to A and waits for a reply. All the conditions for a communications deadlock are now fulfilled.

Source: Answer manual.

Exercise 30

Question

In an electronic funds transfer system, there are hundreds of identical processes that work as follows. Each process reads an input line specifying an amount of money, the account to be credited, and the account to be debited. Then it locks both accounts and transfers the money, releasing the locks when done. With many processes running in parallel, there is a very real danger that a process having locked account x will be unable to lock y because y has been locked by a process now waiting for x . Devise a scheme that avoids deadlocks. Do not release an account record until you have completed the transactions. (In other words, solutions that lock one account and then release it immediately if the other is locked are not allowed.)

Answer

To avoid circular wait, number the resources (the accounts) with their account numbers. After reading an input line, a process locks the lower-numbered account first, then when it gets the lock (which may entail waiting), it locks the other one. Since no process ever waits for an account lower than what it already has, there is never a circular wait, hence never a deadlock.

Source: Answer manual.

Exercise 31

Question

One way to prevent deadlocks is to eliminate the hold-and-wait condition. In the text it was proposed that before asking for a new resource, a process must first release whatever resources it already holds (assuming that is possible). However, doing so introduces the danger that it may get the new resource but lose some of the existing ones to competing processes. Propose an improvement to this scheme.

Answer

Change the semantics of requesting a new resource as follows. If a process asks for a new resource and it is available, it gets the resource and keeps what it already has. If the new resource is not available, all existing resources are released. With this scenario, deadlock is impossible and there is no danger that the new resource is acquired but existing ones lost. Of course, the process only works if releasing a resource is possible (you can release a scanner between pages or a CD recorder between CDs).

Source: Answer manual.

Exercise 32

Question

A computer science student assigned to work on deadlocks thinks of the following brilliant way to eliminate deadlocks. When a process requests a resource, it specifies a time limit. If the process blocks because the resource is not available, a timer is started. If the time limit is exceeded, the process is released and allowed to run again. If you were the professor, what grade would you give this proposal and why?

Answer

I'd give it an F (failing) grade. What does the process do? Since it clearly needs the resource, it just asks again and blocks again. This is no better than staying blocked. In fact, it may be worse since the system may keep track of how long competing processes have been waiting and assign a newly freed resource to the process that has been waiting longest. By periodically timing out and trying again, a process loses its seniority.

Source: Answer manual.

Exercise 33

Question

Main memory units are preempted in swapping and virtual memory systems. The processor is preempted in time-sharing environments. Do you think that these preemption methods were developed to handle resource deadlock or for other purposes? How high is their overhead?

Answer

Both virtual memory and time-sharing systems were developed mainly to assist system users. Virtualizing hardware shields users from the details of prestatating needs, resource allocation, and overlays, in addition to preventing deadlock. The cost of context switching and interrupt handling, however, is considerable. Specialized registers, caches, and circuitry are required. Probably this cost would not have been incurred for the purpose of deadlock prevention alone.

Source: Answer manual.

Exercise 34

Question

Explain the differences between deadlock, livelock, and starvation.

Answer

A deadlock occurs when a set of processes are blocked waiting for an event that only some other process in the set can cause. On the other hand, processes in a livelock are not blocked. Instead, they continue to execute checking for a condition to become true that will never become true. Thus, in addition to the resources they are holding, processes in livelock continue to consume precious CPU time. Finally, starvation of a process occurs because of the presence of other processes as well as a stream of new incoming processes that end up with higher priority than the process being starved. Unlike deadlock or livelock, starvation can terminate on its own, e.g. when existing processes with higher priority terminate and no new processes with higher priority arrive.

Source: Answer manual.

Exercise 35

Question

Assume two processes are issuing a seek command to reposition the mechanism to access the disk and enable a read command. Each process is interrupted before executing its read, and discovers that the other has moved the disk arm. Each then reissues the seek command, but is again interrupted by the other. This sequence continually repeats. Is this a resource deadlock or a livelock? What methods would you recommend to handle the anomaly?

Answer

This dead state is an anomaly of competition synchronization and can be controlled by resource pre-allocation. Processes, however, are not blocked from resources. In addition, resources are already requested in a linear order. This anomaly is not a resource deadlock; it is a livelock. Resource preallocation will prevent this anomaly. As a heuristic, processes may time-out and release their resources if they do not complete within some interval of time, then go to sleep for a random period and then try again.

Source: Answer manual.

Exercise 36

Question

Local Area Networks utilize a media access method called CSMA/CD, in which stations sharing a bus can sense the medium and detect transmissions as well as collisions. In the Ethernet protocol, stations requesting the shared channel do not transmit frames if they sense the medium is busy. When such transmission has terminated, waiting stations each transmit their frames. Two frames that are transmitted at the same time will collide. If stations immediately and repeatedly retransmit after collision detection, they will continue to collide indefinitely.

- (a) Is this a resource deadlock or a livelock?
- (b) Can you suggest a solution to this anomaly?
- (c) Can starvation occur with this scenario?

Answer

Here are the answers, albeit a bit complicated.

- (a) This is a competition synchronization anomaly. It is also a livelock. We might term it a scheduling livelock. It is not a resource livelock or deadlock, since stations are not holding resources that are requested by others and thus a circular chain of stations holding resources while requesting others does not exist. It is not a communication deadlock, since stations are executing independently and would complete transmission were scheduled sequentially.
- (b) Ethernet and slotted Aloha require that stations that detect a collision of their transmission must wait a random number of time slots before retransmitting. The interval within which the time slot is chosen is doubled after each successive collision, dynamically adjusting to heavy traffic loads. After sixteen successive retransmissions a frame is dropped.
- (c) Because access to the channel is probabilistic, and because newly arriving stations can compete and be allocated the channel before stations that have retransmitted some number of times, starvation is enabled.

Source: Answer manual.

Exercise 37

Question

A program contains an error in the order of cooperation and competition mechanisms, resulting in a consumer process locking a mutex (mutual exclusion semaphore) before it blocks on an empty buffer. The producer process blocks on the mutex before it can place a value in the empty buffer and awaken the consumer. Thus, both processes are blocked forever, the producer waiting for the mutex to be unlocked and the consumer waiting for a signal from the producer. Is this a resource deadlock or a communication deadlock? Suggest methods for its control.

Answer

The anomaly is not a resource deadlock. Although processes are sharing a mutex, i.e., a competition mechanism, resource pre-allocation and deadlock avoidance methods are all ineffective for this dead state. Linearly ordering resources is also ineffective. Indeed one could argue that linear orders may be the problem; executing a mutex should be the last step before entering and the first after leaving a critical section. A circular dead state does exist in which both processes wait for an event that can only be caused by the other process. This is a communication deadlock. To make progress, a time-out will work to break this deadlock if it preempts the consumer's mutex. Writing careful code or using monitors for mutual exclusion are better solutions.

Source: Answer manual.

Exercise 38

Question

Cinderella and the Prince are getting divorced. To divide their property, they have agreed on the following algorithm. Every morning, each one may send a letter to the other's lawyer requesting one item of property. Since it takes a day for letters to be delivered, they have agreed that if both discover that they have requested the same item on the same day, the next day they will send a letter canceling the request. Among their property is their dog, Woofers, Woofers's doghouse, their canary, Tweeter, and Tweeter's cage. The animals love their houses, so it has been agreed that any division of property separating an animal from its house is invalid, requiring the whole division to start over from scratch. Both Cinderella and the Prince desperately want Woofers. So that they can go on (separate) vacations, each spouse has programmed a personal computer to handle the negotiation. When they come back from vacation, the computers are still negotiating. Why? Is deadlock possible? Is starvation possible? Discuss your answer.

Answer

If both programs ask for Woofers first, the computers will starve with the endless sequence: request Woofers, cancel request, request Woofers, cancel request, and so on. If one of them asks for the doghouse and the other asks for the dog, we have a deadlock, which is detected by both parties and then broken, but it is just repeated on the next cycle. Either way, if both computers have been programmed to go after the dog or the doghouse first, either starvation or deadlock ensues. There is not really much difference between the two here. In most deadlock problems, starvation does not seem serious because introducing random delays will usually make it very unlikely. That approach does not work here. **PROBLEM SOLUTIONS FOR CHAPTER 7 43**

Source: Answer manual.

Exercise 39

Question

A student majoring in anthropology and minoring in computer science has embarked on a research project to see if African baboons can be taught about deadlocks. He locates a deep canyon and fastens a rope across it, so the baboons can cross hand-overhand. Several baboons can cross at the same time, provided that they are all going in the same direction. If eastward-moving and westward-moving baboons ever get onto the rope at the same time, a deadlock will result (the baboons will get stuck in the middle) because it is impossible for one baboon to climb over another one while suspended over the canyon. If a baboon wants to cross the canyon, he must check to see that no other baboon is currently crossing in the opposite direction. Write a program using semaphores that avoids deadlock. Do not worry about a series of eastward-moving baboons holding up the westward-moving baboons indefinitely.

Answer

Use one mutex to protect the shared counters and one semaphore per direction. The solution below permits any number moving in the same direction and blocks the opposite direction until the rope is empty. This satisfies the exercise's instruction not to worry about starvation.

```
semaphore mutex = 1;
semaphore eastQ = 0, westQ = 0;
int crossing = 0;
int direction = NONE; /* NONE, EAST, or WEST */
int waitingEast = 0, waitingWest = 0;

void enterEast(void) {
    down(mutex);
    if (direction == WEST) {
        waitingEast++;
        up(mutex);
        down(eastQ);
        down(mutex);
        waitingEast--;
    }
    direction = EAST;
    crossing++;
    if (waitingEast > 0) up(eastQ); /* admit another eastbound crosser */
    up(mutex);
}
```

```

void leaveEast(void) {
    down(mutex);
    crossing--;
    if (crossing == 0) {
        direction = NONE;
        if (waitingWest > 0) up(westQ);
    }
    up(mutex);
}

/* enterWest and leaveWest are symmetric. */

```

Expected behavior: eastbound and westbound users are never on the rope at the same time, so deadlock is avoided. Safety note: every counter update must be under `mutex`; otherwise two arrivals can both believe the rope is safe in opposite directions.

Source: Independently prepared technical answer.

Exercise 40

Question

Repeat the previous problem, but now avoid starvation. When a baboon that wants to cross to the east arrives at the rope and finds baboons crossing to the west, he waits until the rope is empty, but no more westward-moving baboons are allowed to start until at least one baboon has crossed the other way.

Answer

To avoid starvation, add a turn variable. Once a waiting process from the opposite direction exists, no new process from the current direction may enter after the rope drains; at least one waiting process from the other direction gets the next turn.

```

semaphore mutex = 1;
semaphore eastQ = 0, westQ = 0;
int crossing = 0, direction = NONE;
int waitingEast = 0, waitingWest = 0;
int turn = EAST;

bool canEnterEast(void) {
    return direction != WEST && !(direction == EAST && waitingWest > 0 &&
    turn == WEST);
}

```



```

void enterEast(void) {
    down(mutex);
    if (!canEnterEast()) {
        waitingEast++;
        up(mutex);
        down(eastQ);
        down(mutex);
        waitingEast--;
    }
    direction = EAST;
    crossing++;
    if (waitingEast > 0 && canEnterEast()) up(eastQ);
    up(mutex);
}

void leaveEast(void) {
    down(mutex);
    crossing--;
    if (crossing == 0) {
        direction = NONE;
        if (waitingWest > 0) { turn = WEST; up(westQ); }
        else if (waitingEast > 0) up(eastQ);
    }
    up(mutex);
}

```

Expected behavior: a stream going one way cannot indefinitely overtake a waiting opposite-direction stream. Safety note: this is pseudocode; in a real implementation use condition variables in a loop so spurious wakeups or missed signals cannot violate the direction rule.

Source: Independently prepared technical answer.

Exercise 41

Question

Program a simulation of the banker's algorithm. Your program should cycle through each of the bank clients asking for a request and evaluating whether it is safe or unsafe. Output a log of requests and decisions to a file.

Answer

The simulation should implement the banker's safety test after each tentative request. A request is granted only if it is no larger than the process's remaining need, no larger than Available, and leaves the system in a safe state.

```
def safe(available, allocation, maximum):
    n, m = len(allocation), len(available)
    need = [[maximum[i][j] - allocation[i][j] for j in range(m)] for i in range(n)]
    work = available[:]
    finish = [False] * n
    order = []
    changed = True
    while changed:
        changed = False
        for i in range(n):
            if not finish[i] and all(need[i][j] <= work[j] for j in range(m)):
                for j in range(m):
                    work[j] += allocation[i][j]
                finish[i] = True
                order.append(i)
                changed = True
    return all(finish), order

def request(pid, req, available, allocation, maximum):
    need = [maximum[pid][j] - allocation[pid][j] for j in range(len(req))]
    if any(req[j] > need[j] for j in range(len(req))):
        return False, "exceeds declared maximum"
    if any(req[j] > available[j] for j in range(len(req))):
        return False, "must wait"
    trial_avail = [available[j] - req[j] for j in range(len(req))]
    trial_alloc = [row[:] for row in allocation]
    for j in range(len(req)):
        trial_alloc[pid][j] += req[j]
    ok, order = safe(trial_avail, trial_alloc, maximum)
    return ok, ("safe order " + str(order) if ok else "unsafe")
```

Expected behavior: the log should show each request, whether it was granted or denied, and the safe completion order when granted. Math note: the key calculation is $\text{Need} = \text{Maximum} - \text{Allocation}$, checked component by component. Safety note: this is a simulator; do not use it as a live resource manager without input validation and atomic updates.

Source: Independently prepared technical answer.

Exercise 42

Question

Write a program to implement the deadlock detection algorithm with multiple resources of each type. Your program should read from a file the following inputs: the number of processes, the number of resource types, the number of resources of each type in existence (vector E), the current allocation matrix C (first row, followed by the second row, and so on), the request matrix R (first row, followed by the second row, and so on). The output of your program should indicate whether there is a deadlock in the system. In case there is, the program should print out the identities of all processes that are deadlocked.

Answer

For multiple resource types, use the deadlock-detection algorithm from the chapter. A process can be marked if its current request vector is componentwise less than or equal to Work. When marked, its allocation is returned to Work.

```
def detect_deadlock(E, C, R):
    n, m = len(C), len(E)
    allocated = [sum(C[i][j] for i in range(n)) for j in range(m)]
    work = [E[j] - allocated[j] for j in range(m)]
    finish = [False] * n
    changed = True
    while changed:
        changed = False
        for i in range(n):
            if not finish[i] and all(R[i][j] <= work[j] for j in
range(m)):
                for j in range(m):
                    work[j] += C[i][j]
                finish[i] = True
                changed = True
    return [i for i, done in enumerate(finish) if not done]
```

```
# File format:
# n m
# E vector
# n rows of C
# n rows of R
```

Expected behavior: if the returned list is empty, no deadlock is detected; otherwise the listed process IDs are deadlocked. Math note: $A = E - \sum_i C_i$ is the initial available vector. Each successful mark adds C_i back to A . Safety note: reject files whose rows have the wrong length or whose allocations exceed E .

Source: Independently prepared technical answer.

Exercise 43

Question

Write a program that detects if there is a deadlock in the system by using a resource allocation graph. Your program should read from a file the following inputs: the number of processes and the number of resources. For each process it should read four numbers: the number of resources it is currently holding, the IDs of resources it is holding, the number of resources it is currently requesting, the IDs of resources it is requesting. The output of the program should indicate if there is a deadlock in the system. In case there is, the program should print out the identities of all processes that are deadlocked.

Answer

Build a directed wait-for graph from the resource-allocation graph and then find directed cycles. For single-instance resources, a process waiting for a resource is waiting for the process that currently holds that resource.

```
def deadlocked_processes(holding, requesting):
    # holding maps resource -> process
    graph = {p: set() for p in requesting}
    for p, reqs in requesting.items():
        for r in reqs:
            owner = holding.get(r)
            if owner is not None and owner != p:
                graph[p].add(owner)

    dead = set()
    def dfs(v, stack, seen):
        if v in stack:
```

```

        dead.update(stack[stack.index(v):])
        return
    if v in seen:
        return
    seen.add(v)
    for w in graph[v]:
        dfs(w, stack + [v], seen)

    for p in graph:
        dfs(p, [], set())
    return sorted(dead)

```

Expected behavior: processes on a directed cycle are reported as deadlocked. Safety note: this graph method is exact for single-instance resources. For multiple instances of one resource type, use the matrix detection algorithm instead because a wait-for cycle alone may be insufficient.

Source: Independently prepared technical answer.

Exercise 44

Question

In certain countries, when two people meet they bow to each other. The protocol is that one of them bows first and stays down until the other one bows. If they bow at the same time, they will both stay bowed forever. Write a program that does not deadlock.

Answer

Break the symmetry by using a mutex or by assigning one participant the right to bow first. The mutex solution ensures that only one pairwise bow protocol is initiated at a time.

```

semaphore bowMutex = 1;

void meet(person other) {
    down(bowMutex);
    bow();
    wait_until(other.has_bowed());
    stand_up();
    up(bowMutex);
}

```

A more concurrent design is to impose a total order on participant IDs: the lower-numbered person bows first and the higher-numbered person responds. The mathematical idea is the

same as resource ordering: because every encounter follows one global order, a circular wait cannot form. Expected behavior: simultaneous meetings no longer leave both participants waiting forever. Safety note: if implemented with messages, add timeout/retry for lost messages; otherwise a communication failure can mimic deadlock.

Source: Independently prepared technical answer.